

Defensive Security Assessment Capstone - Sample Solution

Target Application: OWASP Juice Shop (Educational Vulnerable Web App)

Task 1: Scope, Authorization Statement, & Rules of Engagement

Scope: Testing is strictly limited to the local instance deployed at `http://localhost:3000`. Subdomains, external API endpoints, and production infrastructure are completely out of scope.

Authorization Statement: As the owner and sole administrator of this local environment, full authorization is granted to conduct automated and manual security vulnerability assessments on the specified target application.

Rules of Engagement: All testing will be non-destructive. Denial of Service (DoS) attacks, brute-forcing that locks system resources permanently, and destructive database manipulation are prohibited.

Task 2 & 3: Testing, Findings, & Remediation

1. SQL Injection (SQLi) in Login Component

- *Finding:* The email input field in the login form does not properly sanitize user inputs, allowing an attacker to bypass authentication using a payload like `' OR 1=1 --`.
- *Remediation:* Implement parameterized queries using a secure ORM or prepared statements to ensure user input is treated strictly as data, not executable code.

Secure Code Fix Example (Node.js/Express):

```
models.User.findOne({ where: { email: req.body.email, password:
  hash(req.body.password) } })
```

2. Broken Authentication (Weak Passwords)

- *Finding:* The system allowed registration of simple passwords like 'admin123', making accounts highly vulnerable to credential stuffing and dictionary attacks.
- *Remediation:* Enforce a strict password complexity policy (minimum 8 characters, uppercase, numbers, and special characters) and use strong hashing algorithms.

3. Vulnerable Dependencies (SCA)

- *Finding:* Running `npm audit` revealed a high-severity vulnerability in an outdated version of the `json-jwt` library.
- *Remediation:* Executed `npm install json-jwt@latest` to update the package to a patched version, resolving the high-severity alert.

Task 4: Secure Validation & Logging

- *Implementation:* Configured secure input validation utilizing the express-validator library. Failed authentication attempts are logged without capturing sensitive data like plain-text passwords or personally identifiable information (PII).

Secure Logging Snippet:

```
logger.warn(`Failed login attempt for obfuscated user ID: ${mask(email)} from  
IP: ${req.ip}`);
```

Task 5: Final Documentation Template

To complete your submission, create a README.md file in your public/view-only GitHub repository containing sections for: Executive Summary, Methodology, Vulnerability Details (with reproduction steps), Patches applied (with Git commit hashes), and Retesting confirmation.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.